

THE DESK THAT GRADES ITSELF.

Deterministic code builds every trade. Each analyst carries a track record, states a **probability** rather than a verdict, and is Brier-scored against what actually happened. **An analyst that is confidently wrong loses its vote.**

MOST AGENTS ASK A MODEL WHAT TO BUY. THIS ONE NEVER LETS IT ANSWER.

Asking a model "what should I trade?" is quick to build and impossible to trust. It can hallucinate a strike, size a position wrongly, or be confidently wrong with no record of why.

So the model is handed a **numbered menu it did not write**, and may return one id or the word ABSTAIN. A hallucinated id is treated as an abstention.

- **3,686** contracts read from the live chain
- **1,078** survive the liquidity gate
- **632** defined-risk structures built by code
- **12** shown to the committee
- **1** it may choose, or none

There is no function in this codebase that produces a naked short option.

[FOUR STAGES, EVERY ONE ABLE TO REFUSE]

FOUR CHANCES TO REFUSE. ONE TO TRADE.

- **Build** — code enumerates every legal defined-risk structure, fully priced, before any model is called
- **Argue** — a volatility analyst and a dedicated adversary each return a probability; an abstainer leaves both numerator and denominator
- **Review** — two vetoes that fail differently: pure code checking delta against the stated thesis, and a model that never sees the debate
- **Decide** — RiskGuard returns allow, deny, or a smaller size. Any error, any missing data, any exception is a refusal

Recorded live: the adversary objected to a thin 356-contract hedge, and the trader moved off the highest-credit candidate to answer it. That is the difference between a committee and a rubber stamp.

[WHAT NO COMPETITOR DOES]

IT FIRES ITS OWN ANALYSTS.

Replaying the real committee over 43 post-knowledge-cutoff windows produced 12 resolved predictions per analyst, scored with the standard Brier score:

0.162

VOL_ANALYST BRIER · BETTER

1.18

ITS WEIGHT, UPGRADED

0.298

ADVERSARY BRIER · WORSE

0.90

ITS WEIGHT, DEMOTED

Lower Brier is better. A 0.136 gap is the distance between beating and losing to a coin flip. Weights are recomputed from the journal every cycle and recorded in that cycle's entry, so a judge can see which weights applied to which decision.

Read this honestly. The book was 10-2, and a systematic pessimist scores badly on a winning-skewed sample by construction. The adversary's value is the three vetoes it landed, which Brier does not measure. Twelve outcomes is a first signal, not a verdict. Unproven analysts stay at exactly 1.00; a demoted one floors at 0.20 so it always has a path back.

[OUT-OF-SAMPLE, COMPUTED FROM REAL BARS]

A BACKTEST THAT CANNOT LOSE IS BROKEN.

SYMBOL	TRADES	WIN	EXPECTANCY
SPY	8	87.5%	+0.62R
QQQ	7	85.7%	+0.57R
AAPL	8	62.5%	-0.12R
MSFT	7	85.7%	+0.57R

Thirty trades proves nothing statistically, and one symbol loses money, which is the point. An earlier version of this harness scaled its risk threshold to the wrong horizon and produced a 97% win rate by construction. It was caught and corrected before publication.

The repository this was converted from shipped a hardcoded "82.2% out-of-sample win rate" computed from nothing. That was deleted, not adapted.

Is it reasoning or remembering? The standing criticism of LLM trading agents is knowledge contamination. The model's cutoff is May 2026; live decisions run on August 2026 data. There was nothing to memorise.

[CHECK IT YOURSELF]

DON'T TRUST IT. VERIFY IT.

- `scripts/replay.py --all --verify` reproduces four recorded verdicts offline, no credentials
- `make verify-journal` checks the hash chain
- `make session` runs the full pipeline and sends nothing; submitting needs an explicit flag
- 923 tests, none touching the network

Orders route through the official Alpaca CLI. Market data reads through the official MCP server, observed live with **no order-placement tool exposed at all**.

Two of the four replayable scenarios are **refusals**. The desk declining is the more persuasive half.

Live <https://trading-alpaca-judge.vercel.app>

Judge <https://trading-alpaca-judge.vercel.app/judge>

Smile <https://trading-alpaca-judge.vercel.app/smile>

Code <https://github.com/Prashant-thakur77/trading-alpaca>